Warrant

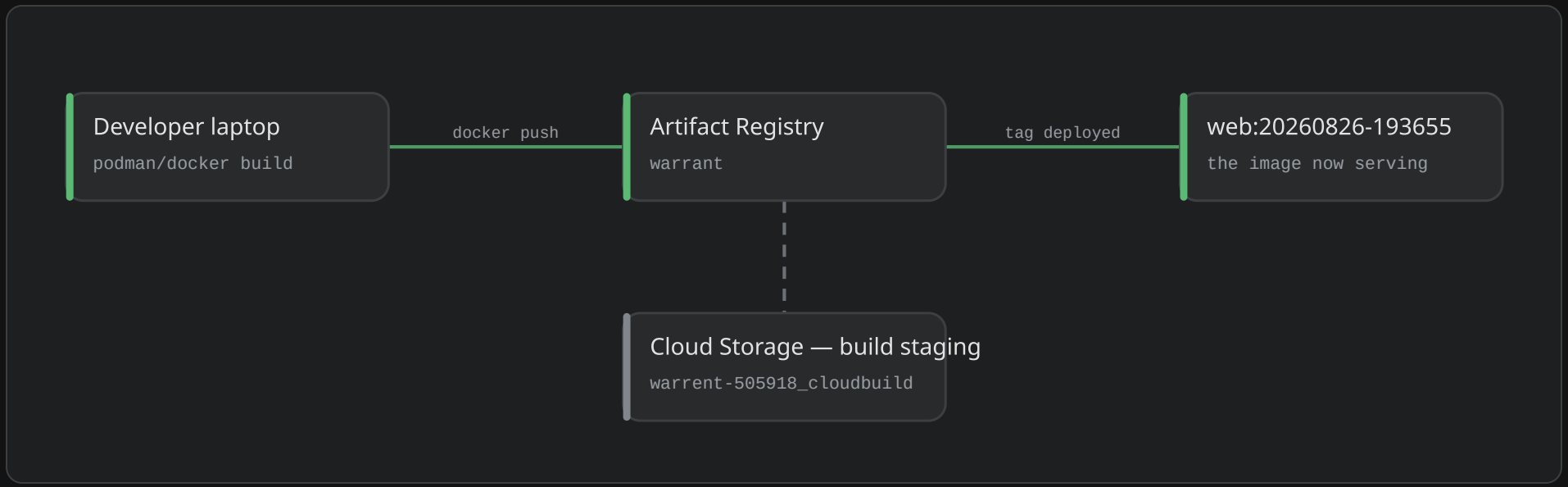
Maintenance records that are evidence, not paperwork. This document describes the system as it is actually deployed, one section at a time. Every value was read out of the live project with gcloud or the REST APIs, or by requesting the running service — not from the repository. Where the two disagree, the project wins and the node is marked drift.

Live · 37 Provisioned · 8 Dormant · 3 Project · 1 On the bench · 3

CONTENTS

Supply chain — how the image got there

The image is built on a laptop and pushed straight to Artifact Registry. Cloud Build is enabled and never runs.



Developer laptop

`podman/docker build · linux/amd64`

ROLE	builds and pushes the image
WHY NOT CLOUD BUILD	new projects have no Cloud Build SA; `builds submit` fails PERMISSION_DENIED
SOURCE	infra/deploy-web.sh

The build host is outside Google Cloud entirely. **Cloud Build is enabled but never runs** — the image is built locally and pushed straight to Artifact Registry.

Artifact Registry

`warrant · Docker · us-central1`

REPOSITORY	warrant
FORMAT	DOCKER
LOCATION	us-central1
IMAGES	web — 8 pushes, 3 tagged
MOST RECENT PUSH	2026-08-26 19:37 UTC

Every deploy this week came through here. Untagged layers accumulate because `deploy-web.sh` tags by timestamp and never prunes.

web:20260826-193655

`the image now serving`

PUSHED	2026-08-26 19:37 UTC
BASE	node:22-alpine
BUILD	Next.js output: standalone
SERVING AS	revision warrant-00013-nlw

Built and deployed **today**. The tag is the build timestamp, so the image name is also the answer to “which commit is live”.

Cloud Storage — build staging

`warrent-505918_cloudbuild · US`

BUCKET	warrent-505918_cloudbuild
LOCATION	US multi-region
CREATED BY	Cloud Build API enablement
USED BY	nothing

Made by enabling an API that never ran. Left in place because deleting it is the sort of tidying that breaks a deploy at midnight.

WHAT THIS SECTION REACHES OUTSIDE ITSELF

`web:20260826-193655` → Revision `warrant-00013-nlw` (Request path)

Identity — Google sign-in, wired to the live origin

The browser never holds a long-lived credential: an ID token is exchanged server-side for an HttpOnly cookie.

Identity Platform

`projects/1020487917587/config`

SERVICE	identitytoolkit.googleapis.com
PROVIDERS	google.com, anonymous
CALLBACK	warrent-505918.firebaseapp.com/__/auth/action
SESSION	minted server-side into an HttpOnly cookie

The browser never holds a long-lived credential. The client SDK gets an ID token, `/api/auth/session` exchanges it, and everything after that is a cookie the page cannot read.

Firebase Web App

Warrant Web · ACTIVE

STATE	ACTIVE
SUPPLIES	the apiKey and authDomain the client SDK needs
STORAGE BUCKET	warrent-505918-evidence

A registration, not a service. It exists so the browser SDK has something to point at.

Google sign-in

enabled · OAuth client present

PROVIDER	google.com
ENABLED	true
CLIENT ID	1020487917587-9s2mkllnpcarhdu4c9uecn3s7rosfks
WHY MANUAL	no public API creates an OAuth client

The one piece of this project that cannot be created by a script. It was clicked once, by hand, in the console.

Anonymous sign-in

enabled

ENABLED	true
USED BY	nothing yet
INTENDED FOR	a technician who joins a job by link

Enabled ahead of the flow that needs it. Nothing in the deployed app calls it.

Authorized domains

4 listed — including the Run host

LISTED	localhost · warrent-505918.firebaseapp.com · warrent-505918.web.app · warrant-zq2l2kwg3q-uc.a.run.app
WAS	3 — the Run host was missing
SYMPTOM WHEN MISSING	the sign-in popup is refused from the live origin

Fixed since the last reading. Sign-in worked on localhost and failed on the deployed site, which is the most expensive class of bug to find on the morning of a demo.

WHAT THIS SECTION REACHES OUTSIDE ITSELF

`Identity Platform` ← `/api/auth/*` (Request path)

`Identity Platform` ← `warrant-web@` service account (Data)

`Authorized domains` → `Google Front End` (Request path)

`Identity Platform` ← `firebase-adminsdk-fbsvc@` (Data)

Request path — what a visitor actually touches

One container serves the pages, the API and the images. Every agent call is made from server code, never from the browser.

Browser

any device · no account, no install

AUTH	Google sign-in, or none for public records
ENTRY	GET /
RESPONSE	200 in 0.41 s warm

The technician surface is a web page on purpose. **A shop cannot be asked to install anything** before it will try this.

Android app

ink.warrant · Compose · debug build

TALKS TO	the same Cloud Run routes as the browser
ADDS	BLE instrument drivers the web cannot reach
DISTRIBUTION	sideloaded — no Play listing
SOURCE	android/app/src/main/java/ink/warrant

The second surface exists for one reason: **Web Bluetooth cannot be trusted to hold a GATT connection while a camera is open**. Everything else it does, the web does.

Google Front End

TLS termination · global anycast

INGRESS	all
HOSTNAMES	warrant-zq2l2kwg3q-uc.a.run.app · warrant-1020487917587.us-central1.run.app
TO CONTAINER	plain HTTP :8080

Nothing was configured here. Cloud Run brings its own certificate and its own edge.

Cloud Run service

warrant · us-central1 · managed

CONCURRENCY	80
SCALE	min 0 · max 4
TIMEOUT	300 s
STARTUP CPU BOOST	on
REVISIONS	13, all ready

Scale-to-zero is why the first request of the morning is slow and every one after it is not. **Max 4 instances is a cost ceiling**, not a capacity estimate.

IAM — run.invoker

allUsers

ROLE	roles/run.invoker
MEMBER	allUsers
EFFECT	the service is open to the public internet

Public by design: a sealed record has to be readable by someone with the link and no account. **Authorisation happens above this line**, in the routes and in firestore.rules.

Revision warrant-00013-nlw

100% traffic · 13th revision

CREATED	2026-08-26 19:37:54 UTC
TRAFFIC	100%, latestRevision
CPU / MEMORY	1 vCPU · 512 MiB
PORT	8080 (http1)
READY	true

Thirteen revisions in seven days. Traffic always follows latest — there is no canary here, and a bad deploy is fixed by deploying again.

Runtime identity

warrant-web@warrent-505918.iam

ATTACHED SA	warrant-web@warrent-505918.iam.gserviceaccount.com
WAS	the default compute SA with roles/editor, until 2026-08-21
CREDENTIALS	metadata server, short-lived
REACHES VERTEX BY	impersonating warrant-adjudicator@

Fixed since the last reading of this diagram. The service ran as the default compute account — project editor — for its first two revisions. It now runs as a purpose-made account that cannot administer the project it lives in.

Next.js standalone server

node:22-alpine · server.js on :8080

MODE	output: standalone
PROCESS	node server.js, PID 1
STATIC	/_next/static served in-process
ROUTES	22 API routes · 17 pages

One container serves the pages, the API and the images. There is no separate backend, and **every agent call in this system is made from server code**, never from the browser.

LiveSource

Firestore-backed · fabricated = false

BINDING	getDataSource() → LiveSource
REASON	GCP_PROJECT and the Firebase env are baked into the image
WAS	FixtureSource, until the env was set on 2026-08-21
FALLBACK	FixtureSource still runs the whole product offline

Fixed since the last reading. The deployed image used to fall back to fixtures because no configuration reached it, and the site said so out loud. It now reads and writes the real database.

Public routes

7 checked, all 200

CHECKED	2026-08-26, live service
200	/ · /library · /model-tests · /about · /author · /fleet · /records
RENDERER	React Server Components + client islands
SLOWEST	/fleet, 1.87 s — it queries the engine

Every page a judge is asked to open was requested from the public internet on the day this diagram was made.

/api/auth/*

session · claim · calendar

/API/AUTH/SESSION	200
/API/AUTH/CLAIM	405 — POST only, as written
WAS	all 404 — absent from the image on 2026-08-19
ALSO	/api/auth/calendar for the OAuth grant

Fixed since the last reading. These routes existed only in the working tree at the time of the last deploy, so sign-in could not complete against the live service.

WHAT THIS SECTION REACHES OUTSIDE ITSELF

Revision warrant-00013-nlw ← web:20260826-193655 (Supply chain)

LiveSource → Firestore (default) (Data)

Google Front End ← Authorized domains (Identity)

Cloud Run service → 51 APIs enabled (Platform services)

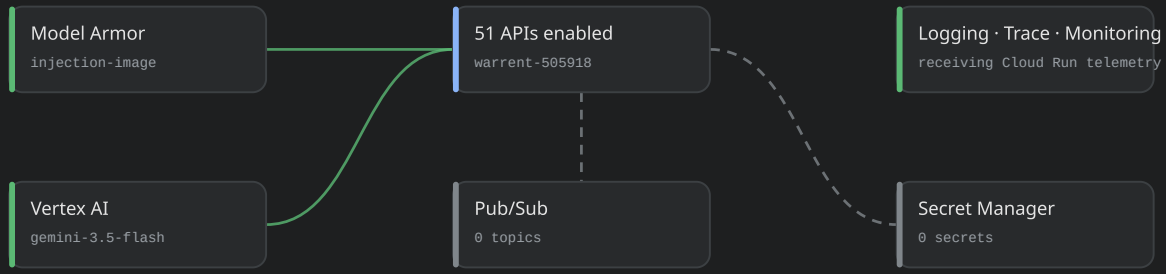
Revision warrant-00013-nlw → Logging · Trace · Monitoring (Platform services)

/api/auth/* → Identity Platform (Identity)

Public routes → Vertex AI Agent Engine (The fleet)

Platform services

What the project actually calls, and what was enabled and left alone. Both are drawn, because an unused service is still a fact about the build.



Model Armor

injection-image · locations/us

TEMPLATE	projects/warrent-505918/locations/us/templates/injection-image
CREATED	2026-08-18
FILTERS	prompt injection · jailbreak · RAI DANGEROUS at LOW_AND_ABOVE
WIRED BY	MODEL_ARMOR_LOCATION + MODEL_ARMOR_TEMPLATE on the revision

Now in the request path. Every capture is screened before a judgement model is asked about it, because **a photograph is untrusted input** and a sticker with words on it is a prompt.

Vertex AI

gemini-3.5-flash · global endpoint

JUDGEMENT MODEL	gemini-3.5-flash
SCREENING MODEL	gemini-3.5-flash-lite
ENGINE-SIDE	gemini-3.5-flash-lite, observed serving 2026-08-26
LOCATION	global — the model is not where the rest of this runs
TEMPERATURE	0.0

Two tiers, chosen by **what it costs to be wrong**: Flash-Lite refuses an unusable photo, Flash judges the evidence, and nothing cheap can pass a step. **The screen is not Gemma** — Gemma is not a publisher model on Vertex and every spelling of it 404s, so running it would mean paying for a GPU endpoint that has to still be up when a judge watches the film.

51 APIs enabled

warrent-505918

ENABLED	51
WAS	46 on 2026-08-19
ENABLED BY	infra/bootstrap.sh, once

Most of these are transitive — enabling Firebase pulls in a dozen. The ones this system actually calls are drawn.

Pub/Sub

0 topics

TOPICS	0
SUBSCRIPTIONS	0
ENABLED BY	bootstrap.sh

Enabled and never used. The sweep is a cron, not a queue — see the scheduled band.

Logging · Trace · Monitoring

receiving Cloud Run telemetry

LOGGING	request and stdout logs
TRACE	spans named agent.inspector, agent.skeptic, screen — the screen runs Flash-Lite
MONITORING	Run built-in metrics
GAP	the app logs no stack on a 500

The agent calls are traced by hand, so a slow adjudication can be read span by span. **Application errors are not logged**, which is how the broken sweep stayed quiet.

Secret Manager

0 secrets

SECRETS	0
WHERE THE SECRETS ARE INSTEAD	plain env vars on the revision
AT RISK	the sweep secret, the OAuth client secret, the instrument keys

The honest weak point of this deployment. Cloud Run env vars are visible to anyone with describe on the service. For a hackathon build that is a considered trade; for a real fleet it is the first thing to move.

WHAT THIS SECTION REACHES OUTSIDE ITSELF

Logging · Trace · Monitoring ← Revision warrant-00013-nlw (Request path)

Vertex AI ← The cheap screen (The fleet)

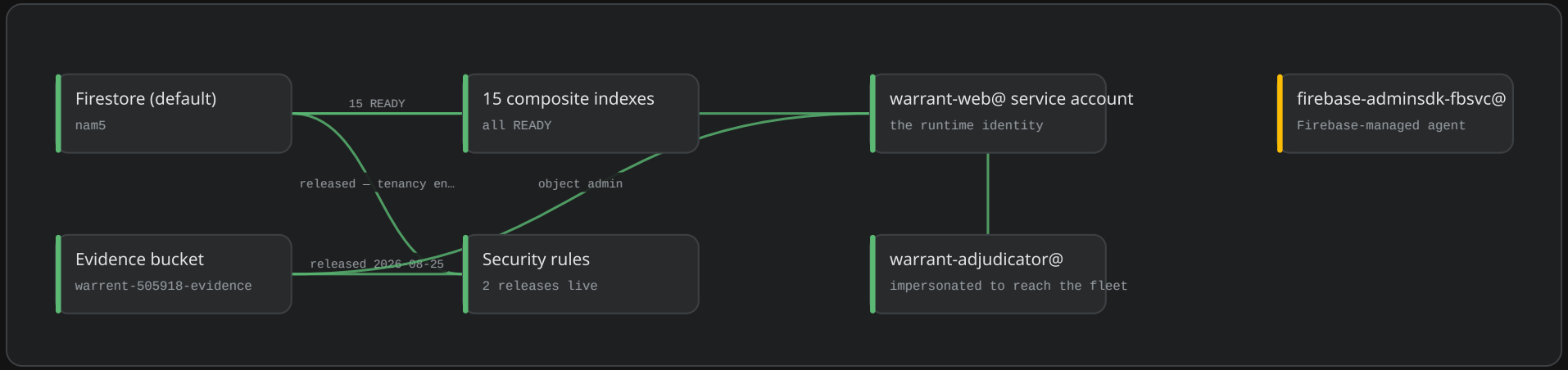
51 APIs enabled ← Cloud Run service (Request path)

Vertex AI ← Vertex AI Agent Engine (The fleet)

Model Armor → Vertex AI Agent Engine (The fleet)

Data — carrying real records

Everything lives under tenants/{t}/..., so a query that forgets its tenant returns nothing rather than everything.



Firestore (default)

nam5 · Native · pessimistic

TENANTS	22
JOBS	35
PROCEDURES	56 (+1 published publicly)
SEALED RECORDS	2
MEMBERS	21

No longer empty. Everything is under `tenants/{t}/…` so a query that forgets its tenant returns nothing rather than everything. The two sealed records are the ones the demo was cut from.

15 composite indexes

all READY

COMPOSITE INDEXES	15
WAS	7 on 2026-08-19
DEPLOYED BY	infra/deploy-rules.sh

Every one of these exists because a query failed once. Firestore names the index it wants in the error, which makes this the least interesting file in the repository and the most annoying to omit.

Security rules

2 releases live

CLOUD.FIRESTORE	released 2026-08-25 14:11 UTC
WARRENT-505918-EVIDENCE	released 2026-08-25 14:11 UTC
WAS	2 rulesets uploaded, neither released
PROVED BY	the emulator, in scripts/smoke.sh

Fixed since the last reading. Rules that are uploaded but never released do nothing at all, and the console shows them as if they were live. Tenancy is now enforced by the database, and smoke.sh proves it by running the rules rather than asserting

Evidence bucket

warrent-505918-evidence · 52 objects

OBJECTS	52
LOCATION	US-CENTRAL1
HOLDS	capture media, avatars, uploaded paper forms
RULES	released 2026-08-25

The photographs themselves. **There was no storage rule at all until 2026-08-20**, while the bucket was already configured — the default posture must not be discovered on the day the first photograph is uploaded.

warrant-web@ service account

the runtime identity

ATTACHED TO	the Cloud Run revision
HOLDS	datastore.user · firebaseauth.admin · storage object admin on the evidence bucket
DOES NOT HOLD	project editor
CAN	impersonate warrant-adjudicator@

Least privilege, and now actually in use. It cannot administer the project, cannot create service accounts, and cannot read another project's data.

warrant-adjudicator@

impersonated to reach the fleet

ROLE	aiplatform.user, and nothing else
REACHED BY	warrant-web@ minting a short-lived token
WHY SPLIT	the web identity should not hold a standing key to the model

The split exists so that **the credential that can call a judgement model is not the credential serving web pages**. It is issued for minutes at a time and never written down.

firebase-adminsdk-fbsvc@

Firebase-managed agent

CREATED BY	Firebase, automatically
USED BY	nothing in this deployment

Comes with the Firebase registration. Left alone.

The fleet — deployed on Vertex AI Agent Engine

Seven agents behind one engine, which answers for its own roster over the API — so the claim is checkable from outside.

Vertex AI Agent Engine

`warrant-fleet · reasoningEngines/5032906174249304064`

DISPLAY NAME	warrant-fleet
CREATED	2026-08-21 22:16 UTC
LAST UPDATED	2026-08-25 16:03 UTC
METHODS	query · roster · screen
ROSTER ANSWERS	seven agents, each with its contract

The fleet is deployed, not on a laptop. Asked for its roster over the API it names all seven agents and the contract each one answers under — so the claim on this diagram is checkable from outside by anyone with the endpoint.

The cheap screen

`gemini-3.5-flash-lite · not an eighth agent`

ANSWERS	UNUSABLE or NEEDS_JUDGEMENT — there is no PASS
COSTS WHEN WRONG	one retake
NOT IN	REGISTRY — roster() says seven
WHY NOT GEMMA	not a publisher model on Vertex — every spelling returns 404
TRACED AS	screen

The cheap model sits only on the side where being wrong is recoverable. **It has no PASS in its enum**, so the strongest answer it can give still ends with the judge or the technician being asked. That is a shape,

Agent package bucket

`warrent-505918-agents · 13 objects`

OBJECTS	13
HOLDS	the packaged fleet the engine runs
WRITTEN BY	infra/deploy-agents.py

How the code in `agents/` gets to Vertex. The engine pulls from here at deploy, not at request time.

Scoper

`13/14 · interviews, then compiles`

DECIDES	what to ask next, and when a procedure would run unambiguously
CONTRACT	scoper-turn.schema.json
SCENARIOS	14, including full interviews — all recorded
REACHED BY	/api/scoper/turn and /api/procedures/compile
REFUSES	to state a tolerance nobody gave it

There is no form builder in this product and this agent is the reason. Its hardest scenario runs a whole interview against a shop played by a second model, then checks **every bound in the compiled procedure against the numbers the shop actually**

Inspector

`12/15 · one field at a time`

DECIDES	PASS · ADD FIELD · ESCALATE on one field's evidence
CONTRACT	inspector-verdict.schema.json
SCENARIOS	15, every photograph taken
REACHED BY	/api/adjudicate
NEVER JUDGES	a whole step — it would trade a weak photo against a strong one

Reads the photograph against the written acceptance rule, and where it falls short **composes the specific next request** rather than asking for a generic retry. Numeric bounds are not its job: `within(min,max)` is deterministic code, and the…

Skeptic

`10/12 · adversarial`

DECIDES	does this evidence belong to this job, this machine, this moment
CONTRACT	skeptic-verdict.schema.json
SCENARIOS	12, every photograph taken
REACHED BY	/api/adjudicate, beside the Inspector
NEVER SEES	the Inspector's verdict — its prompt does not mention an Inspector exists

Given the perceptual embedding distance as an *input*: code computes the distance, the model decides what it means against the machine's recorded damage and history. Twelve identical bikes

Instructor

`6/8 · unbounded speech in`

DECIDES	the blocker class, the next action, and the safety flag
CONTRACT	instructor-recommendation.schema.json
SCENARIOS	8
REACHED BY	the disposition path, when a step stalls
SEES	the technician's words verbatim, untidied

"I haven't got the right socket" and "the socket won't bite, it's rounded" share a vocabulary and mean **tool_missing** versus **seized_or_damaged** — which decides whether the next visit needs an extractor or

Foreman

`8/8 · owns the job for its life`

DECIDES	status, action, and whether to recommend holding the machine
CONTRACT	foreman-disposition.schema.json
SCENARIOS	8
REACHED BY	the disposition path and the sweep
CANNOT	hold anything itself — the Gate does, deterministically

Its inputs conflict on purpose: the machine is unsafe, the customer is booked in two days, and the torque wrench arrives Thursday. It writes down enough to be woken days later with no other context. **A waiver needs a named person holding standing**, and…

Auditor

5/5 · weeks of jobs at once

DECIDES	whether the PROCEDURE is defective, and hands the revision back
CONTRACT	auditor-finding.schema.json
SCENARIOS	5
REACHED BY	the sweep, every minute
REFUSES	to say what a wrong bound should become

The only agent whose subject is the procedure, and the one that closes the loop back to the Scoper. Its hard judgement is that **a step that keeps failing is usually the procedure working** — pads worn past the limit on nine jobs in twenty is a rule catching re…

Wright

7/7 · writes the driver

DECIDES	which characteristic carries a reading, and how it is encoded
CONTRACT	wright-turn.schema.json
SCENARIOS	7, all offline from recorded GATT trees
REFUSES	a number whose unit it cannot name

The only agent that emits an artefact rather than a verdict, which is why it is held hardest. A wrong verdict is on the record to argue with; **a wrong driver mints wrong measurements unattended**, each filed as *measured*. So it must never pick batter…

WHAT THIS SECTION REACHES OUTSIDE ITSELF

Vertex AI Agent Engine ← warrant-adjudicator@ (Data)

The cheap screen → Vertex AI (Platform services)

Vertex AI Agent Engine ← Public routes (Request path)

Wright → Driver on the phone (Instruments)

Vertex AI Agent Engine → Vertex AI (Platform services)

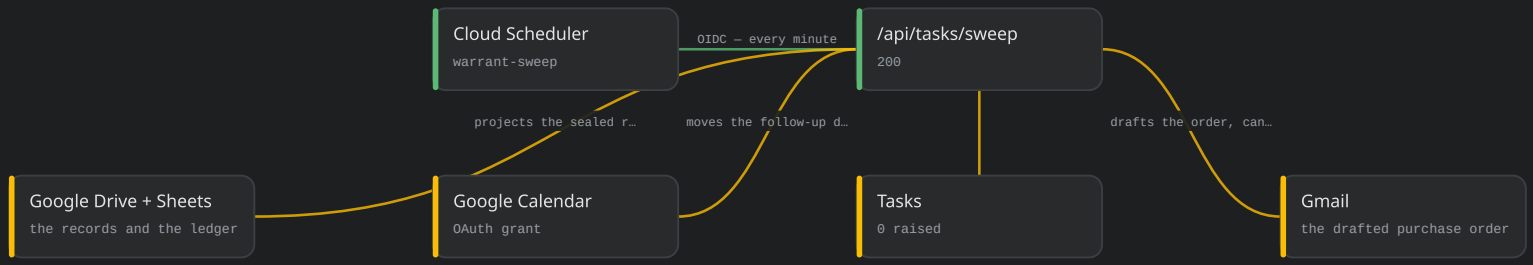
Vertex AI Agent Engine ← Model Armor (Platform services)

Foreman → Firestore (default) (Data)

Auditor ← /api/tasks/sweep (Scheduled work)

Scheduled work — one cron for the whole system

One cron re-reads the world every minute, so nothing is lost when a deploy lands mid-flight. It failed for a day and was fixed in revision 00014.



Cloud Scheduler

`warrant-sweep · * * * * *`

SCHEDULE	* * * * * — every minute
WAS	*/5, until deploy-web.sh reset it
STATE	ENABLED
TARGET	POST /api/tasks/sweep
AUTH	OIDC token

One cron for the whole system. Cloud Tasks would be more precise; a cron that re-reads the world cannot lose work when a deploy lands mid-flight.

/api/tasks/sweep

`200 · 3–11 s`

STATUS	200 on 2026-08-27, revision 00014
WAS	500 on every run for a day, across revisions 00012 and 00013
RUNTIME	10.6 s cold, 3.0 s warm
UNAUTHENTICATED	401, correctly
HOLDS	a lease, so a long run is not overtaken by its own retry

This was the one red node on this diagram, and it is fixed. For a day the cron authenticated, threw early and returned 500 in under a second — before any model call, on two revisions, so no single deploy caused it. Revision 00014 answers 200 and completes in seconds. Two consecutive runs observed; the collections it writes are still empty, which is a separa…

Tasks

`0 raised`

TASKS	0
RAISED BY	the Foreman and the Auditor, through the sweep
NOTE	the sweep now completes, so an empty table is a real answer

Still empty — but the sweep above now runs, so this is no longer explained by a broken cron. Two sealed records is a thin window for an Auditor whose subject is a procedure across weeks of jobs; there is genuinely little for it to find yet.

Google Calendar

`OAuth grant · /api/auth/workspace`

SCOPE	calendar.events — write only
WRITTEN BY	the sweep
IDEMPOTENCY	extendedProperties.warrant_task_id, so a re-run updates rather than duplicates
EVENTS CREATED	none against real Google

A held machine that needs a part on Thursday should put Thursday in somebody's calendar. One of three APIs behind a single Workspace grant. Wired and granted; not yet exercised against Google.

Google Drive + Sheets

`the records and the ledger`

SCOPE	drive.file — per-file access to files THIS APP created
PER SEAL	one Doc in the shop's folder, one row in their ledger
LEDGER	a Sheet, on the same scope — Sheets accepts drive.file for a sheet the app made
IDS AT	/tenants/{t}/integrations/workspace, server-written
FOUND BY	an unfiltered rotating window over records — no index needed

The record leaves the building. A maintenance record is worth something years later, often to somebody who never had a login here — so the seal is projected into the shop's own Drive. Warrant is still the source of truth and this is a copy: nothing reads it back, and a shop that deletes the folder loses a convenience, not a record.

Gmail

`the drafted purchase order`

SCOPE	gmail.compose — creates a draft, CANNOT send, cannot read the mailbox
WRITTEN BY	the Foreman, through dispose()
LANDS IN	a foreman's drafts, never the technician's
IDEMPOTENCY	the draft id on the task, not a header search — Gmail's q does not index X- headers
NOTIFICATION MAIL	a separate identity: the tenant's notifier mailbox, keyless via IAM signJwt

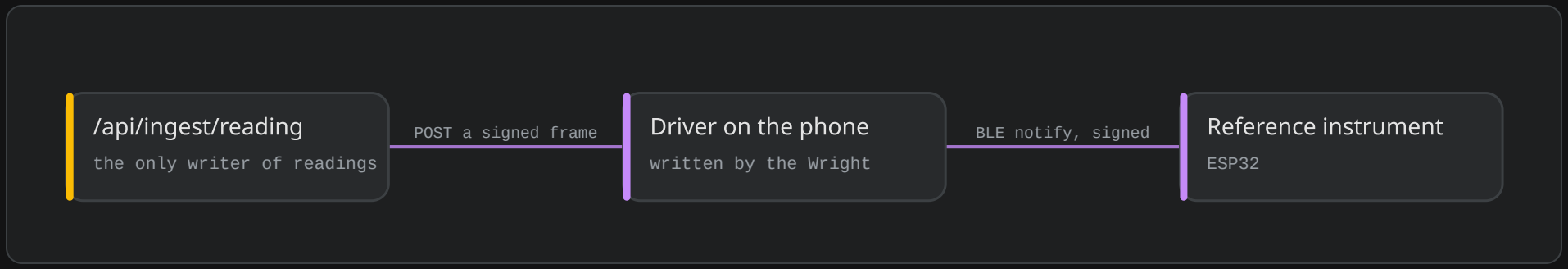
Autonomy stops where money leaves the business, and the credential is what stops it. An agent that decided to order forty thousand pounds of steel could not transmit the order — the scope cannot send. 'Drafted, never sent' is a property of the grant rather than an instruction a model was asked to follow.

WHAT THIS SECTION REACHES OUTSIDE ITSELF

`/api/tasks/sweep` → Auditor (The fleet)

Instruments — a reading nobody typed

A signed frame from an instrument is the difference between a measurement and a number somebody typed.



Reference instrument

ESP32 · BLE · signed frames

TRANSPORT	BLE GATT notify
SIGNS	every frame, with a counter persisted in NVS
WHY A COUNTER	BLE is a broadcast — without it a frame can be replayed
SOURCE	firmware/warrant_reference_instrument.ino

Real hardware on a bench, not a simulation. It exists to prove the shape: **a torque wrench that signs its readings** means the number on the record was measured rather than typed.

Driver on the phone

written by the Wright

READS	the GATT tree, then the characteristic the Wright chose
RUNS ON	the Android surface only
FALLS BACK TO	a typed value, recorded as unattested

The phone is the bridge. An instrument that cannot be read is not a blocker — the reading is simply recorded as one a person typed, and the record says so.

/api/ingest/reading

the only writer of readings

READINGS WRITTEN	0 in production
RULES	refuse `readings` from every client
UNSIGNED READING	accepted, but recorded with no tool_id
KEYS	WARRANT_INSTRUMENT_KEYS, compared in constant time

The only writer of `readings`, and that is the point of the endpoint. No client may write a measurement directly, so "measured" means a signed frame reached this route — and an unsigned one still gets recorded, just without the claim.

WHAT THIS SECTION REACHES OUTSIDE ITSELF

`Driver on the phone` ← `Wright` (The fleet)

`/api/ingest/reading` → `Firestore (default)` (Data)

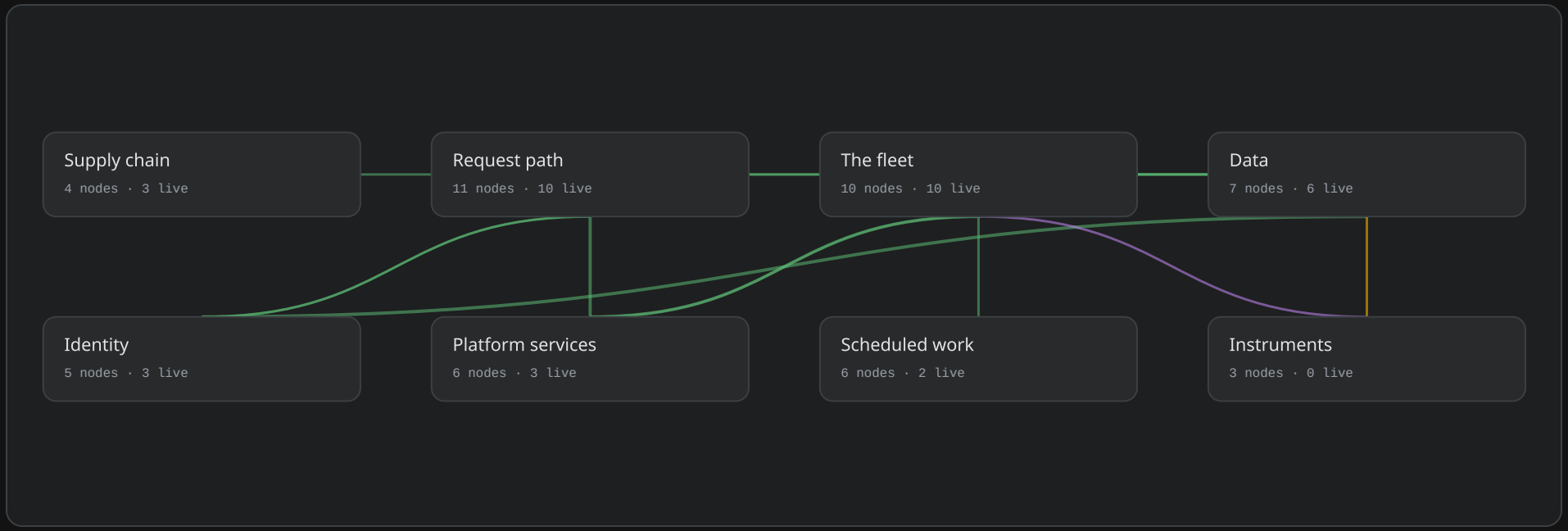
HOW THEY COME TOGETHER

One capture, end to end

The sections above are the parts. This is the path a single photograph takes through all of them, in order, on the deployed system.

1. A technician opens the job on the web or the Android app. There is nothing to install, and a sealed record is readable by anyone with the link — which is why the service is public to `allUsers`.
2. Google Front End terminates TLS and hands plain HTTP to the Cloud Run service `warrant`, one container serving the pages, the API and the images.
3. The revision runs as `warrant-web@` — not the default compute account it used to run as — so the identity serving web pages cannot administer the project.
4. The capture is written to the evidence bucket and the job to Firestore, both under `tenants/{t}/…`, with released rules refusing anything that reaches across a tenant.
5. **Model Armor screens the photograph first.** A photograph is untrusted input, and a sticker with words on it is a prompt.
6. The cheap screen (Flash-Lite) asks one question: is this frame so unusable that spending the judge on it is waste? It has no PASS in its enum, so it can cost a retake and nothing more.
7. `warrant-web@` mints a short-lived token for `warrant-adjudicator@` and calls `reasoningEngines:query`. The credential that can reach a judgement model is never the one serving pages.
8. The Inspector judges the evidence against the written rule; the Skeptic asks whether it belongs to this job at all. **Neither sees the other's verdict** — two agents shown each other's conclusions agree, and the second opinion stops being one.
9. The verdict lands on the record with the model and the version that produced it. The Gate — deterministic code, not an agent — decides whether the step passes and whether the machine is held.
10. Every five minutes Cloud Scheduler calls the sweep, which is where the Auditor reads weeks of jobs at once and hands a procedure defect back to the Scoper.

THE EIGHT SECTIONS, AND THE TRAFFIC BETWEEN THEM



Step 10 was broken for a day, and is not any more. The sweep returned 500 on every run across two revisions, failing in under a second — before any model call — so no audit ran against real data. Revision 00014 answers 200 and completes in three to eleven seconds. `tasks`, `findings` and `audits` are still empty, which is now a thinness-of-data question rather than a broken one: two sealed records is a narrow window for an Auditor whose subject is a procedure across weeks of jobs.